

GAMESMANROS 2 · ONBOARDING

How it works, and where to start

A mechanical walkthrough of the ROS 2 system — how a move actually becomes robot motion, how each subsystem is built, and the specific places three people can start contributing this week.

Companion to the architecture deck. That one covered *why* we rewrote the system — this one covers *how* it works and where the open work is.

Your first twenty minutes in the repo

Do not read the codebase front to back. There are seven nodes but only one path through them that matters. Open these four files in this order — it is about 600 lines total and it is the whole system.

ORDER	FILE	WHAT TO TAKE FROM IT	SKIP ON THE FIRST PASS
1	game_manager.py	The loop. Where moves come from and how they get sent.	The lifecycle callbacks — come back to those
2	robot_control.py	The translator. Move string → board coordinates → motion goals.	Type2 / Type6 / Type7 — read Type4 only
3	arm_controller.py	The planner. Where MoveIt 2 and OMPL are actually called.	The quaternion composition at the top
4	centers.py	The data. Pure tables, no ROS — the easiest file to read.	Nothing, it is 89 lines

The mental model

Seven processes that only talk through typed messages. No shared memory, no imports across node boundaries. If you change one node's internals, nothing else can notice.

The one rule

The `.action` and `.msg` files are the contracts between tracks. Changing one affects everybody, so those changes go through Ethan. Everything else is yours to refactor.

Before you debug anything

Check the broken-list slide near the end. Several things are known-broken right now. Do not spend an afternoon discovering one of them and assuming you caused it.

How a ROS 2 action actually works



Why this matters here

Motion takes seconds and can fail. The old system published a command and assumed it worked. With an action, `robot_control` learns whether the arm actually reached the target — and a failed plan propagates all the way back to the game loop.

Topic vs service vs action

Topic — fire and forget, many listeners, no reply.

Service — request/reply, but blocks and cannot report progress.

Action — request, progress, cancel, and a final result. The right tool for anything slow.

Seeing it live

`ros2 action list` shows all three of ours. `ros2 action send_goal /move_arm ...` lets you drive the arm by hand with no game running — the fastest way to test anything.

One move, traced through real code

```
# game_manager.py – pick and send
move_str = _pick_best_move(moves)      # "A_3_7"
goal.game_id      = "dodgem"
goal.move_string = "A_3_7"

# robot_control.py – decode and dispatch
async def _execute(goal_handle):
    inst = _get_game_instance(req.game_id)
    await inst.process_move("A_3_7", positions)

# Type6.process_move – split the string
parts = move.split('_')      # ['A','3','7']
await seq.pick_and_place(conv, centers[3], centers[7])

# MotionSequencer – 9 sequential goals
bx,by,bz = conv.svg_to_meters(before, 135.0)
await _send_gripper(open_=True)
await _send_arm(bx, by, lz)    # hover
await _send_arm(bx, by, bz)    # descend
await _send_gripper(open_=False) # grip
```

Where the string is decoded

UWAPI move strings are `type_from_to`. Everything downstream is just `split('_')` and two integer indices into the `centers` table. There is no parser and no game logic in the robot code — the solver already decided.

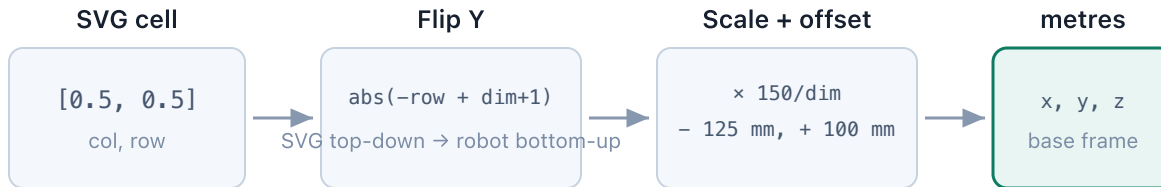
Where it becomes motion

One game move expands into **nine action goals** — two gripper, seven arm. Each one is awaited, so a failure at step 3 stops steps 4 through 9 via the `ok = ok` and `await ...` chain.

The trick worth understanding

`await` on an `rcpy Future` hands control back to the executor instead of blocking the thread. That is what lets one node run an action server and two action clients at the same time without deadlocking.

Board square → point in space



Worked examples, from the real tables

dodgem cell 0 [0.5, 0.5]	→ x=-0.100 y= 0.275 z= 0.135
dodgem cell 8 [0.5, 2.5]	→ x=-0.100 y= 0.175 z= 0.135
1dchess cell 4 [4.5, 0.5]	→ x=-0.041 y= 0.259 z= 0.135

Two z values exist: 135 mm to grip, 185 mm to travel. Never travel at grip height.

Why a Y flip

SVG counts rows downward from the top-left. The robot's Y axis increases away from its base. Without the flip, every board would be mirrored front to back.

Where the constants live

`BOARD_SIZE_MM = 150`, `Y_OFFSET_MM = 100`, `PICKUP_Z_MM = 135`, `LIFT_Z_MM = 185` — all at the top of `robot_control.py`. These are physical measurements of the board's position on the table and will need re-measuring once the robot is mounted.

This is where the bugs are

The conversion code is correct. The *data it reads* is not — six of nine games have inconsistent coordinate tables. That is Track C's first job, and it is covered three slides on.

What happens when you ask for a pose

```
# 1. Build the target pose
pose.position    = (x, y, z)
pose.orientation = _EEF_ORIENTATION # fixed

# 2. Hand it to MoveIt
_arm.set_start_state_to_current_state()
_arm.set_goal_state(
    pose_stamped_msg = pose,
    pose_link = 'joint6_flange')

# 3. Plan — IK, then OMPL search
plan_result = _arm.plan()
if not plan_result:
    goal_handle.abort() # caller finds out

# 4. Execute the timed trajectory
_moveit.execute(plan_result.trajectory,
                blocking=True)
```

Four steps, and step 3 is the one the whole rewrite was about.

Step 3a — inverse kinematics

MoveIt first has to find six joint angles that put the flange at your `(x, y, z)` with the required orientation. That is IK, solved by KDL. Not every reachable-looking point has a solution at a fixed orientation — near the workspace edge, many do not.

Step 3b — OMPL path search

With a valid goal configuration, RRTConnect grows two trees — one from the current joint angles, one from the goal — until they meet. It searches the six-dimensional joint space, not 3D space, and checks collisions against the URDF along the way.

Why the orientation is frozen

The gripper always points down, rotated 45° about Z with an 11.25° wrist correction. Fixing it removes wrist-flip solutions and makes planning repeatable — but it also shrinks the reachable set. If a board square turns out to be unreachable, this constraint is the first suspect.

Four markers, one matrix, no calibration

```
# Find the four board-corner markers
corners, ids, _ = detector.detectMarkers(frame)
id_map[i] = corners[i][0].mean(axis=0)

# Map them onto a flat square
src = [id_map[0], id_map[1],
       id_map[2], id_map[3]]      # TL TR BR BL
dst = [[0,0], [600,0],
       [600,600], [0,600]]

H = cv2.getPerspectiveTransform(src, dst)
warped = cv2.warpPerspective(frame, H, (600,600))

# Classify each cell by hue
n1 = cv2.countNonZero(cv2.inRange(cell, p1_lo, p1_hi))
if n1 / cell_px > 0.15: occupancy.append('1')
```

Why no camera calibration

Four points with known destinations fully determine a projective transform. Camera angle, height and tilt all get absorbed into `H`. Move the camera mid-game and the next frame still rectifies correctly — the markers are re-detected every frame.

Why HSV and not RGB

Hue stays roughly constant as lighting changes; RGB values do not. Thresholding on hue means a red piece reads as red in a bright room and a dim one. The saturation and value floors exist to reject grey shadows.

Two real limits

The grid is assumed uniform `rows × cols`, so irregular boards like `dinododgem` are not handled. And a homography models a plane, not lens distortion — cells near the frame edge drift slightly. Both are open work.

Track A — MoveIt 2 bring-up

The critical path. **There is no ROS 2 MoveIt configuration in the repo at all** — only ROS 1 URDFs under `catkin_ws/`. Until one exists, `arm_controller` cannot start, so nothing can be validated on hardware.

You do not need the robot

MoveIt runs against mock hardware. You can build the config, launch RViz and confirm OMPL plans a valid trajectory with nothing plugged in. That is the whole first milestone.

First task

Port the MyCobot 280 URDF into a ROS 2 `moveit_config` package and get RViz to display the arm. No motion yet.

Done when: `ros2 launch <your_config> demo.launch.py` shows the arm and you can drag the interactive marker.

What the config must define

- Planning group named `arm_group`
- Tip link `joint6_flange`
- Self-collision matrix (SRDF)
- OMPL as the default planning pipeline
- A `joint_trajectory_controller`

Those two names are hardcoded in `arm_controller.py`. Match them or change both.

Blocked on, and blocking

You are blocked by: nothing. Start today.

You block: all hardware validation for the whole team. Flag slips early.

Owns

new `moveit_config` package
`arm_controller.py`

Track B — Perception

Fully independent of the robot and of Track A. The pipeline is written but has never seen a real camera frame — the HSV bounds in the launch file are generic red/blue placeholders.

Most of this is not even ROS

The calibration work is plain OpenCV on a laptop webcam. Write and tune it as a normal Python script, then move the finished numbers into the node. Docker on macOS cannot see USB cameras, so do this part natively.

First task

Print the four ArUco markers (`DICT_4X4_50` , IDs 0–3), point a webcam at a mock board, and get `vision_node` publishing any board state at all.

Done when: `ros2 topic echo /vision/board_state` prints a string that changes when you move a piece.

The work after that

- An HSV calibration tool, so bounds can be tuned without editing code
- Non-uniform grid support — `1dchess` is 8×1, `dinododgem` is irregular
- Detecting a human's move from board-state changes

Already exposed as parameters

```
p1_hsv_lower / p1_hsv_upper  
p2_hsv_lower / p2_hsv_upper  
grid_rows / grid_cols  
camera_index / debug_view
```

Set `debug_view:=true` for a live annotated window showing the rectified board and per-cell classification.

Owns

```
vision_node.py  
new calibration tooling
```

Track C — Game data and test harness

No hardware, no ROS knowledge needed to start. `centers.py` is pure Python data — and an audit of it turned up a systematic bug affecting **six of nine games**.

GAME	DIM	COORD CONVENTION	IMPLIED GRID	PROBLEM
1dchess	8	raw cell units	8	consistent
dinododgem	6	raw cell units	6	consistent
joust	4	raw cell units	4	consistent
dao	4	normalised 0–1	—	board comes out 4× too small
allqueenschess	5	normalised 0–1	—	board comes out 5× too small
dragonsandswans	missing	normalised 0–1	—	no dim → <code>TypeError</code> , and unregistered
dodgem	3	raw cell units	4	dim disagrees with grid → wrong spacing
jan	3	raw cell units	4	dim disagrees with grid → wrong spacing
dawsonschess	5	raw cell units	6	dim disagrees, and needs <code>_pickup</code>

First task

Fix the empty `_pickup` table so `dawsonschess` stops raising `TypeError` on its first move. Smallest, most concrete item on the board.

Then

Pick one convention for all nine games and convert the rest. Add the seven registered games that have no coordinates at all. Decide whether `dragonsandswans` is registered or dropped.

Then

A UWAPI mock so the stack runs while the solver API is down, and a CLI harness that fires `ExecuteMove` goals directly. The whole team will use both.

What is already broken

Verified by building and running the workspace in the dev container. If you hit one of these, you did not cause it.

BLOCKS HARDWARE

`mycobot_interfaces` is missing

`joint_state_bridge.py` and `gripper_controller.py` both import `MycobotAngles` / `MycobotGripperStatus` from a package that is not in this repo and is not a stock ROS 2 package. **Two of seven nodes cannot start.** We need to vendor Elephant Robotics' `mycobot_ros2` or define equivalent messages ourselves.

Currently unassigned.

Six of nine games have bad coordinates

Mixed normalised and raw conventions, plus `dim` values that disagree with the actual grid. See the Track C slide.

No MoveIt 2 configuration exists

`arm_controller` imports fine but cannot run without it. Track A's first deliverable.

Launch ordering is unenforced

Every node starts simultaneously, so `arm_controller` can come up before `/joint_states` is being published.

Zero tests in the repository

Nothing to regress against. Whoever builds the first harness sets the pattern for everyone.

Already fixed — do not re-break

NumPy is pinned below 2.0 in the Dockerfile because `tf_transformations` calls a function NumPy 2 removed. OpenCV comes from pip, not apt, because Ubuntu's 4.5.4 lacks `cv2.aruco.ArucoDetector`.

The commands you will actually use

BUILD AND RUN

```
# open a dev shell (from repo root)
docker compose -f docker/docker-compose.yml \
  run --rm dev

# inside the container
colcon build --symlink-install
source install/setup.bash

# run one node
ros2 run gamesmanros robot_control

# run the system
ros2 launch gamesmanros gamesmanros.launch.py \
  vision:=true
```

`--symlink-install` means Python edits apply without rebuilding. Editing `.action` or `.msg` files still needs a real rebuild.

INSPECT AND DEBUG

```
# what is running
ros2 node list
ros2 action list
ros2 topic list

# watch the board state
ros2 topic echo /vision/board_state

# drive the arm by hand – no game needed
ros2 action send_goal /move_arm \
  gamesmanros_interfaces/action/MoveArm \
  "{x: -0.10, y: 0.275, z: 0.185}"

# open and close the gripper
ros2 action send_goal /move_gripper \
  gamesmanros_interfaces/action/MoveGripper \
  "{open: true}"
```

`send_goal` is the fastest way to test anything. It bypasses the game loop entirely, so you can exercise motion with UWAPI down.

Full setup instructions, per-track environment notes and troubleshooting are in `docs/SETUP.md`. Verify your environment with the four checks in that file before the next meeting.